DANIEL SEVERO DOS SANTOS - RU: 5837105 **Nota: 100**

PROTOCOLO: 2026092158371059417 69C

Disciplina(s):

Banco de Dados Relacional

de início:	9/2026 18:25
o máximo entrega:	
de entrega:	9/2026 18:44

Atenção. Este gabarito é para uso exclusivo do aluno e não deve ser publicado ou compartilhado em redes sociais ou grupo de mensagens.

O seu compartilhamento infringe as políticas do Centro Universitário UNINTER e poderá implicar sanções disciplinares, com possibilidade de desligamento do quadro de alunos do Centro Universitário, bem como responder ações judiciais no âmbito cível e criminal.

Questão 1/15 - Banco de Dados Relacional

Ler em voz alta

As *subqueries* são uma ferramenta muito poderosa para trabalhar com consultas SQL (*Structured Query Language*), permitindo uma ampla variedade de possibilidades e ampliando as informações que estão sendo avaliadas.

Nesse contexto, assinale a alternativa verdadeira sobre a utilização de *subqueries* em uma consulta SQL (*Structured Query Language*).

Nota: 6.7

A *Subqueries* não podem ser usadas com cláusulas *where* ou *having* na consulta externa.

B

Subqueries precisam estar escritas fora dos parênteses na consulta externa.

C

A *subquery* mais interna não tem precedência na execução da consulta externa.

D

***Subqueries* são usadas na consulta externa para ampliar o conjunto de valores analisados através dos filtros *where* ou *having*.**

Você assinalou essa alternativa (D)

Você acertou!

As *subqueries* são usadas na consulta principal, também chamada de consulta externa, para fornecer dados adicionais que são usados como entrada nas cláusulas *where* ou *having* na consulta externa.

Referência: Aula 5 - Comandos Avançados; Tema 2 - Formatação de dados textuais; *Subqueries*.

E

Subqueries precisam retornar um valor único na consulta externa.

Questão 2/15 - Banco de Dados Relacional

Ler em voz alta

Em um Banco de Dados ocorre a execução de múltiplas transações simultaneamente, o que pode provocar diversos problemas de concorrência.

Com base nessa informação, considere o seguinte cenário: A transação T1 lê um dado que sofreu alteração pela transação T2. Porém, a transação T2 ainda não confirmou a modificação.

Assinale a alternativa que contém a nomenclatura desse tipo de problema.

Nota: 6.7

A Leitura fantasma.

B Leitura defeituosa.

C Leitura não repetida.

D Leitura duplicável.

E **Leitura suja.**

Você assinalou essa alternativa (E)

Você acertou!

Quando uma transação acessa um dado que foi alterado por outra transação, mas que ainda não foi confirmado (*commit*), ocorre o que é chamado de leitura suja, uma vez que a transação está manipulando uma informação que pode estar incorreta.

Referência: Aula 6 - Procedimentos Armazenados; Tema 1 - Índice e visão (*index* e *view*); Transações.

Questão 3/15 - Banco de Dados Relacional

Ler em voz alta

A cláusula *order by* é usada em consultas SQL (*Structured Query Language*) para classificar os resultados em ordem crescente ou decrescente.

Com base nessa informação, analise o seguinte enunciado:

Considere uma tabela "Pedido", a qual contém as seguintes colunas: "pedidoId", "clienteId", "produtoId", "dataPedido" e "valorPedido".

Assinale a alternativa em que cláusula *order by* é utilizada para classificar a tabela "Pedido" pelo valor do pedido em ordem crescente e, em caso de valores iguais, pela data do pedido em ordem decrescente.

Nota: 6.7

A

select * from Pedido order by valorPedido desc, dataPedido;

B

select * from Pedido order by valorPedido, dataPedido desc;

Você assinalou essa alternativa (B)

Você acertou!

Nesta consulta, a cláusula *order by* é usada para classificar a tabela "Pedido" pelo valor do pedido (valorPedido) em ordem crescente e, em caso de valores iguais, pela data do pedido (dataPedido) em ordem decrescente.

A cláusula *order by* segue a ordem das colunas especificadas na consulta. Logo, a ordem do valor do pedido vem antes da ordem da data do pedido. Isso significa que os valores do pedido serão classificados primeiro e, em caso de valores iguais, a data do pedido será usada como critério de desempate para classificar os resultados. Vale

salientar que a classificação padrão é ascendente (*asc*) e, dessa forma, sua declaração é opcional.

Referência: Aula 4 - Manipulação de Estruturas de Dados; Tema 3 - Restrição de consultas; *Order by*.

C

select * from Pedido order by valorPedido desc, dataPedido desc;

D

select valorPedido, dataPedido from Pedido order by valorPedido desc, dataPedido;

E

select * from Pedido order by valorPedido, dataPedido;

Questão 4/15 - Banco de Dados Relacional

Ler em voz alta

Em um Banco de Dados relacional, as *triggers* são procedimentos armazenados que são acionados automaticamente quando uma determinada ação ocorre em uma tabela, como inserção, atualização ou exclusão de dados. Há duas formas de definir as *triggers* (padrão SQL ANSI), em nível de linha e em nível de instrução.

Assinale a alternativa que descreve corretamente as diferenças entre os dois tipos de *triggers*.

Nota: 6.7

A

Uma *trigger* em nível de linha é executada uma vez para cada linha afetada por uma instrução, enquanto uma *trigger* em nível de instrução é executada apenas uma vez por instrução.

Você assinalou essa alternativa (A)

Você acertou!

Uma *trigger* em nível de linha é executada uma vez para cada linha que é afetada por uma instrução. Por exemplo, ao atualizar cinco linhas em uma tabela com uma *trigger* em nível de linha, a *trigger* será acionada cinco vezes, uma vez para cada linha afetada.

No caso da *trigger* em nível de instrução, a mesma é executada apenas uma vez por instrução. Por exemplo, ao atualizar cinco linhas em uma tabela com uma *trigger* em nível de instrução, a *trigger* será acionada apenas uma vez, independentemente do número de linhas afetadas.

Referência: Aula 6 - Procedimentos Armazenados; Tema 2 - *Trigger*; *Trigger*.

B

Uma *trigger* em nível de linha é executada apenas uma vez por instrução, enquanto uma *trigger* em nível de instrução é executada uma vez para cada linha afetada por uma instrução.

C

Ambas as *triggers*, em nível de linha e em nível de instrução, são executadas uma vez para cada linha afetada por uma instrução.

D

Ambas as *triggers*, em nível de linha e em nível de instrução, são executadas apenas uma vez por instrução, independentemente do número de linhas afetadas.

E

Uma *trigger* em nível de linha é executada antes de qualquer outra *trigger* em nível de instrução, enquanto uma *trigger* em nível de instrução é executada depois de todas as *triggers* em nível de linha.

Questão 5/15 - Banco de Dados Relacional

Ler em voz alta

Em MySQL, uma função é uma rotina de Banco de Dados que retorna um valor e pode ser executada como parte de uma instrução *select* ou uma expressão.

Nesse contexto, assinale a alternativa que define corretamente a *query* de criação de uma função que retorna a soma de dois valores inteiros.

Nota: 6.7

A

```
create function Somar (in a int, in b int) returns int
begin
    return a + b;
end;
```

B

```
create function Somar (in a int, in b int)
begin
    returns int a + b;
end;
```

C

```
create function Somar (a int, b int) returns int
begin
    return a + b;
end;
```

Você assinalou essa alternativa (C)

Você acertou!

A *query* apresentada define corretamente a criação de uma função em MySQL, função intitulada "Somar". Tal função aceita dois parâmetros do tipo *int* (inteiros), "a" e "b". O comando *returns int* especifica que a função retornará um valor do tipo *int* (inteiro), isto é, o resultado da adição dos dois parâmetros "a" e "b". O comando *begin* define o início do corpo da função, que contém apenas um comando, o comando *return*, que retorna o resultado da soma dos dois parâmetros. O comando *end* define o fim da função. Ao ser

invocada, a função "Somar" recebe dois valores inteiros e retorna o resultado da soma como um valor inteiro.

Referência: Aula 6 - Procedimentos Armazenados; Tema 4 - *Function*; *Function*.

D

```
create function Somar (a int, b int) returns int;
    return a + b;
end;
```

E

```
create function Somar (in a int, in b int)
    return a + b;
end;
```

Questão 6/15 - Banco de Dados Relacional

Ler em voz alta

O processo de eliminação de linhas em uma tabela é importante para remover informações desatualizadas ou irrelevantes da base de dados. No entanto, é importante ter cuidado ao realizar esse processo, pois a eliminação de linhas pode afetar a integridade dos dados, colocando em risco aspectos de consistência, precisão e validade dos dados armazenados na base de dados.

Com base nessa informação, avalie as alternativas listadas e assinale a opção que identifica os cuidados que devem ser tomados durante o processo de eliminação de linhas em uma tabela, a fim de garantir a integridade dos dados.

Nota: 6.7

A

Verificar se a tabela tem todas as restrições desabilitadas antes da eliminação.

B

Verificar se a tabela tem todas as chaves estrangeiras desabilitadas antes da eliminação.

C

Verificar se não existem outras tabelas dependentes da tabela em questão antes da eliminação.

Você assinalou essa alternativa (C)

Você acertou!

A eliminação de linhas pode afetar a integridade dos dados em outras tabelas que dependem da tabela em questão. Por isso, é importante verificar antes se existem outras tabelas dependentes e, se necessário, realizar a eliminação dessas tabelas primeiro.

Referência: Aula 4 - Manipulação de Estruturas de Dados; Tema 2 - Exclusão e modificação de registros; Removendo linhas da tabela.

D

Verificar se a tabela tem todas as restrições habilitadas antes da eliminação.

E

Verificar se a tabela tem todas as chaves estrangeiras habilitadas antes da eliminação.

Questão 7/15 - Banco de Dados Relacional

Ler em voz alta

O conceito de visão, também conhecido como tabela virtual, é um objeto que retorna um conjunto de dados de uma consulta armazenada em um Banco de Dados. Tal recurso é muito útil e bastante empregado na implementação de Banco de Dados.

Assinale a alternativa correta que destaca as aplicações do conceito de visão.

Nota: 6.7

A

Ocultar dados e fornecer o controle de redundâncias.

B

Simplificar o acesso às tabelas relacionadas e isolar a estrutura da tabela de uma aplicação.

Você assinalou essa alternativa (B)

Você acertou!

Ao se criar uma visão que envolve uma consulta com diversas tabelas, a visão irá simplificar esse acesso, fornecendo a visualização de uma única tabela para o usuário, ou seja, ocultando as tabelas (estrutura lógica) envolvidas na consulta.

Referência: Aula 6 - Procedimentos Armazenados; Tema 1 - Índice e visão (*index* e *view*); *View*.

C

Fornecer controle de redundância e restrição de integridade.

D

Fornecer controle de redundância e independência da estrutura lógica dos dados.

E

Disponibilizar informações referentes a tabela.

Questão 8/15 - Banco de Dados Relacional

Ler em voz alta

O comando *insert* é utilizado para incluir informações de dados, adicionando uma nova linha a uma tabela de um Banco de Dados. Tal comando é uma das operações mais básicas de manipulação de dados e é amplamente utilizado.

Avalie as alternativas a seguir e assinale a que representa a sintaxe completa e correta do comando *insert*.

Nota: 6.7

A

insert into Nome_Tabela set coluna1 = valor1, ..., colunaN = valorN;

B

insert Nome_Tabela (coluna1, ..., colunaN) set valor1, ..., valorN;

C

insert into Nome_Tabela (coluna1, ..., colunaN) values (valor1, ..., valorN);

Você assinalou essa alternativa (C)

Você acertou!

A sintaxe completa do comando *insert* é definida da seguinte forma: declara-se o comando propriamente dito, *insert*, acompanhado da palavra-chave *into*, a qual é usada para especificar o nome da tabela a ser manipulada, ou seja, define o destino dos dados a serem inseridos. Logo após, declara-se o nome da tabela (Nome_Tabela), as colunas em que se deseja incluir dados (coluna1, ..., colunaN), seguida pela palavra-chave *values* (para definição dos valores) e, finalmente, os respectivos valores correspondentes (valor1, ..., valorN) a cada coluna da sequência da declaração.

Referência: Aula 4 - Manipulação de Estruturas de Dados; Tema 1 - Inclusão de registros; Inclusão de registros.

D

insert Nome_Tabela set (coluna1, ..., colunaN) values (valor1, ..., valorN);

E

insert from Nome_Tabela (coluna1, ..., colunaN) values (valor1, ..., valorN);

Questão 9/15 - Banco de Dados Relacional

Ler em voz alta

Em Banco de Dados, uma *function* é, basicamente, um segmento de código ou rotina SQL (*Structured Query Language*) que recebe parâmetros de entrada, processa as instruções definidas e retorna ao ponto de chamada com um resultado. Além das vantagens de reutilização de código, facilidade no processo de manutenção e abstração de rotinas complexas de cálculo e lógica, sua aplicação também é importante em situações onde o MySQL não oferece suporte (de forma nativa) a determinadas situações.

Nesse contexto, sem levar em consideração o uso do delimitador de comando (*delimiter*), assinale a alternativa que apresenta a forma correta de criação de uma *function*.

Nota: 6.7

A

declare nomeFuncao ([parametro1 tipoDado, ..., parametroN tipoDado]) returns tipoDado

begin

codigoFuncao

end;

B

create function nomeFuncao ([parametro1 tipoDado, ..., parametroN tipoDado]) returns tipoDado

begin

codigoFuncao

end;

Você assinalou essa alternativa (B)

Você acertou!

As palavras-chave *create function* são usadas para criar uma nova função no Banco de Dados. Logo após essas palavras, é fornecido o nome da função, que poderá conter, entre parênteses, uma lista de parâmetros (opcional), isso se a função precisar de entrada. O próximo passo é declarar a palavra-chave *returns*, que é usada para especificar o tipo de dado que a função retornará. Finalmente, o corpo da função é definido entre as palavras-chave *begin* e *end*. É nesse local que se escreve o código da função, a qual pode incluir instruções *select*, *if*, *while*, entre outras.

Referência: Aula 6 - Procedimentos Armazenados; Tema 4 - *Function*; Declaração de uma função.

C
```
declare nomeFuncao() returns tipoDado
begin
    codigoFuncao
end;
```

D
```
create function nomeFuncao() returns tipoDado
begin
    codigoFuncao
end;
```

E
```
declare nomeFuncao ([parametro1 tipoDado, ..., parametroN tipoDado])
begin
    codigoFuncao
end;
```

Questão 10/15 - Banco de Dados Relacional

Ler em voz alta

A formatação de dados numéricos é uma técnica utilizada para exibir dados numéricos em um formato específico, de acordo com as necessidades do usuário ou do projeto.

Assinale a alternativa que descreve como a função *mod*() atua em Banco de Dados relacional.

Nota: 6.7

A

Divide dois números e retorna o resultado inteiro.

B

Retorna o resultado da divisão de dois números.

C

Retorna o resto da divisão de dois números.

Você assinalou essa alternativa (C)

Você acertou!A função *mod*() tem como objetivo retornar o valor correspondente ao resto de

uma divisão. Por exemplo, "mod(4, 2)" é equivalente a dividir o valor 4 pelo valor 2, o que resultará no valor 0 (zero), sendo, nesse caso, o resultado fornecido pela função *mod*().

Referência: Aula 5 - Comandos Avançados; Tema 3 - Formatação de dados numéricos e temporais; *Mod*() e *div*().

D

Arredonda o resultado da divisão de dois números.

E

Retorna o resultado da multiplicação de dois números.

Questão 11/15 - Banco de Dados Relacional

Ler em voz alta

O *alias* é um comando que permite renomear uma tabela ou uma coluna. Assim, é utilizado para simplificar o nome das tabelas e/ou colunas, além de reduzir a escrita dos comandos, pois algumas vezes, definir um nome mais amigável a uma tabela ou coluna, torna mais fácil e intuitivo identificar os dados.

Analise as alternativas listadas e assinale a que apresenta a forma correta de utilizar o comando *alias* em *scripts* SQL (*Structured Query Language*).

Nota: 6.7

A

select coluna1 alias temporaria_col1, coluna2 alias temporaria_col2 from Nome_Tabela tab1;

B

select coluna1 as temporaria_col1, coluna2 as temporaria_col2 from Nome_Tabela;

Você assinalou essa alternativa (B)

Você acertou!

O comando *alias* em *Structured Query Language* (SQL) é uma forma de dar um apelido ou um nome temporário a uma tabela ou coluna, ou seja, diferente dos nomes reais das tabelas ou colunas na base de dados.

O *script*

select coluna1 as temporaria_col1, coluna2 as temporaria_col2 from Nome_Tabela;

Possui a sintaxe correta para criar um *alias*, cuja palavra-chave é *as*, seguida do nome do apelido desejado, que, nesse caso, são "temporaria_col1" e "temporaria_col2".

Referência: Aula 4 - Manipulação de Estruturas de Dados; Tema 5 - Outros comandos e recomendações; *Alias* (apelido das tabelas e colunas).

C

select coluna1 & temporaria_col1, coluna2 & temporaria_col2 from Nome_Tabela tab1;

D

select coluna1 to temporaria_col1, coluna2 to temporaria_col2 from Nome_Tabela;

E

select coluna1 = temporaria_col1, coluna2 = temporaria_col2 from Nome_Tabela;

Questão 12/15 - Banco de Dados Relacional

Ler em voz alta

O comando *insert* é usado para inserir novos dados em uma tabela. Quando se está inserindo valores em uma coluna, é possível especificar um valor *null* para atribuição a coluna.

Com base nessa informação, o que acontecerá ao inserir um valor *null* em uma coluna com valor padrão definido?

Nota: 6.7

A

O valor *null* será inserido corretamente.

Você assinalou essa alternativa (A)

Você acertou!

O valor *null* será inserido corretamente, pois o mesmo é considerado pelo Banco de Dados um valor indefinido, isto é, a coluna em questão não terá um valor definido, mas terá o valor *null*. Logo, o valor padrão só é inserido se não for especificado nenhum valor.

Referência: Aula 4 - Manipulação de Estruturas de Dados; Tema 1 - Inclusão de registros; Inclusão de registros.

B

Ocorre um erro de violação de chave estrangeira.

C

Ocorre um erro de sintaxe.

D

O valor inserido será o valor padrão definido para a coluna.

E

Ocorre um erro de violação de chave primária.

Questão 13/15 - Banco de Dados Relacional

Ler em voz alta

A função *count*() é uma função de agregação da *Structured Query Language* (SQL), a qual é utilizada para contar o número de linhas em uma tabela. Tal função é frequentemente usada, em combinação com o comando *select*, para retornar o número de linhas que atendem a determinadas condições.

Analise e assinale a alternativa que corresponde ao resultado da execução da seguinte *query*:

select count(*), count(generoFunc) from Funcionario;

Nota: 6.7

A

A primeira contagem retorna o número de linhas únicas da tabela "Funcionario", enquanto a segunda contagem retorna o número total de linhas.

B

A primeira contagem retorna o número total de linhas da tabela "Funcionario", enquanto a segunda contagem retorna o número de linhas nulas da coluna "generoFunc".

C

A primeira contagem retorna o número de linhas distintas da tabela "Funcionario", enquanto a segunda contagem retorna o número de linhas nulas da coluna "generoFunc".

D

A primeira contagem retorna o número de linhas nulas da tabela "Funcionario", enquanto a segunda contagem retorna o número total de linhas da tabela.

E

A primeira contagem retorna o número total de linhas da tabela "Funcionario", enquanto a segunda contagem retorna o número de linhas que não são nulas da coluna "generoFunc".

Você assinalou essa alternativa (E)

Você acertou!

A primeira contagem da *query* retorna o número total de linhas da tabela "Funcionario", o que é feito através do uso do operador * dentro da função *count*(), que conta todas as linhas da tabela, independentemente do valor da coluna.

A segunda contagem, por sua vez, retorna o número de linhas não nulas da coluna "generoFunc". A função *count*() conta apenas as linhas que não são nulas, ou seja, ignora as linhas nulas. Dessa forma, a segunda contagem retorna o número de linhas da coluna "generoFunc" que possuem valores válidos (não nulos).

Referência: Aula 5 - Comandos Avançados; Tema 4 - Agregação/Extração de dados; *Count*().

Questão 14/15 - Banco de Dados Relacional

Ler em voz alta

Para ampliar o potencial dos filtros utilizados em consultas SQL (*Structured Query Language*), faz-se uso dos operadores lógicos, aritméticos e de comparação. Tais expressões são utilizadas para filtrar informações e selecionar apenas os dados desejados,

podendo, inclusive, serem combinadas de várias maneiras para formar expressões mais complexas.

Entre as alternativas listadas abaixo, assinale a opção equivalente a expressão "X > 5 and X < 10".

Nota: 6.7

A **not (X <= 5 or X >= 10).**

Você assinalou essa alternativa (A)

Você acertou!

A expressão "X > 5 and X < 10" significa que *X* é maior que 5 e menor que 10 ao mesmo tempo. Logo, a resposta correta é "not (X <= 5 or X >= 10)", que representa a negação da condição de que *X* é menor ou igual a 5 ou maior ou igual a 10. Em outras palavras, a expressão "not (X <= 5 or x >= 10)" é equivalente a *X* ser maior que 5 e menor que 10, ou seja, equivalente a expressão original "X > 5 and X < 10".

Referência: Aula 4 - Manipulação de Estruturas de Dados; Tema 3 - Restrição de consultas; Operadores lógicos e operadores relacionais.

B 5 < X > 10.

C 5 > X and X < 10.

D X <= 5 and X <= 10.

E 5 >= X or X >= 10.

Questão 15/15 - Banco de Dados Relacional

Ler em voz alta

Um *cursor* é uma estrutura que permite percorrer e manipular linhas em um conjunto de resultados retornados por uma consulta SQL (*Structured Query Language*), podendo acessar o conjunto de linhas retornadas e processar cada um desses conjuntos.

Assinale a alternativa que lista o comando que realiza essa função.

Nota: 6.7

A

***Fetch*.**

Você assinalou essa alternativa (A)

Você acertou!

O comando *fetch* é utilizado em cursores em Banco de Dados para recuperar linhas específicas a partir da posição atual do cursor, lendo a linha do cursor que está alocada na memória.

Referência: Aula 6 - Procedimentos Armazenados; Tema 5 - *Cursor*; *Cursor*.

B

Open.

C

Close.

D

Repeat.

E

Read.